

メディアコンピューティング2026

第2回 画像・動画の入出力

デジタル画像の基礎

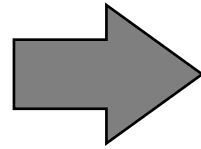
■一般的には二次元格子構造

➤格子：**画素**

■各格子に明るさや色を示す数値が入る

➤**画素値**

5	12	100	188	.	.	.	210
35	200	97	121	.	.	.	65
128	0	21	72	.	.	.	172
76	220	119	97	.	.	.	132
.	.	.	.				.
.	.	.	.				.
.	.	.	.				.
100	29	98	43	.	.	.	87



デジタル画像の基礎

■画像サイズ

➤横方向画素数 (cols) × 縦方向画素数 (rows)

◆4:3・・・640×480 (VGA), 800×600 (SVGA),
1024×768 (XGA) など

◆16:9・・・1280×720 (HD), 1920×1080 (Full HD),
3840×2160 (4K) など

◆16:10・・・1280×800 (WXGA), 1920×1200 (WUXGA) など

■色深度

➤画素値を表すビット数

◆画素値の階調

◆8ビット (256階調), 14ビット (16384階調) など

デジタル画像の基礎

■チャンネル数

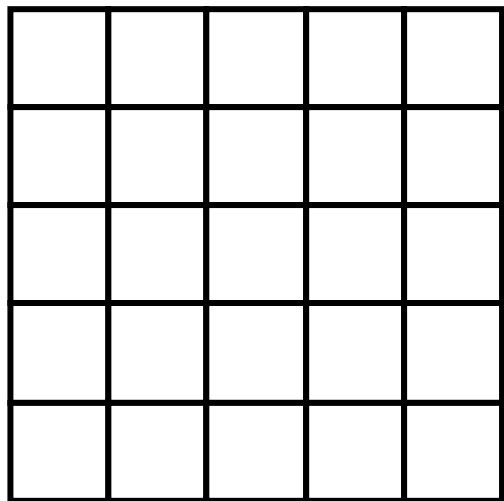
- グレースケール画像：1チャンネル
- カラー画像：3チャンネル，4チャンネル
 - ◆4チャンネル目：透明度を表す α チャンネル

■カラー画像の色空間

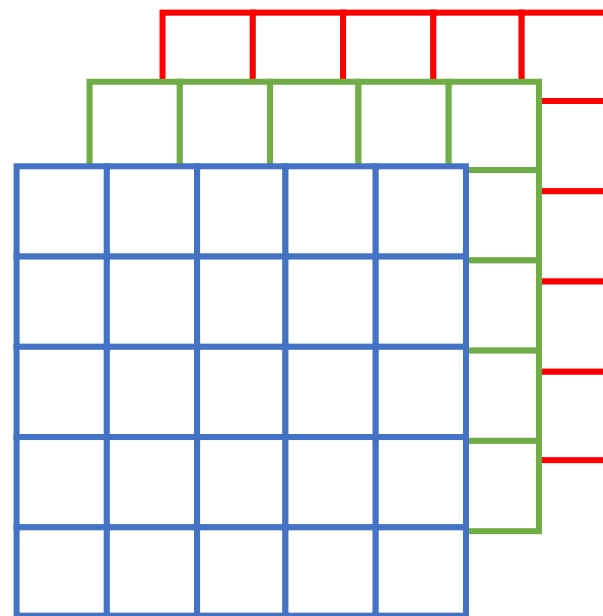
- RGB・・・R（赤），G（緑），B（青）
- HSV・・・H（色相），S（彩度），V（明度）
- CMYK・・・C（シアン），M（マゼンタ），Y（イエロー），
K（キー（黒））

デジタル画像の基礎

■ グレースケール及びカラーのデジタル画像の構造



グレースケール画像
： 1 チャンネル



カラー画像
： 3 チャンネル

色深度が 8 ビット → 各画素値は0~255の256階調
(3チャンネル→ $256^3=16,777,216$ 階調)

デジタル画像とRGB

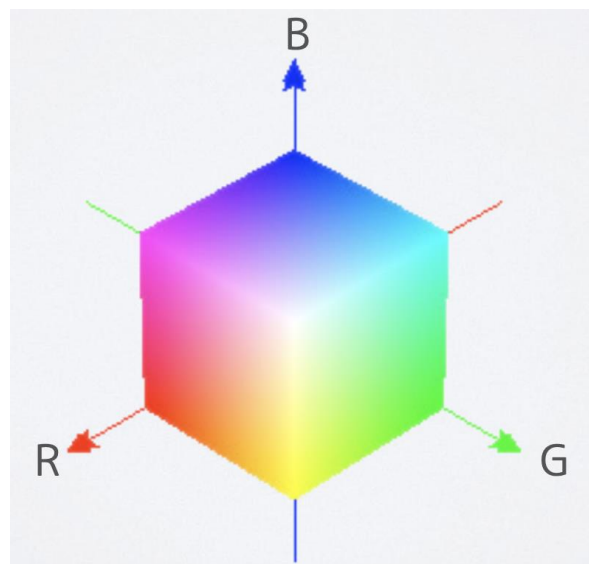
■RGB（順番はBGRの場合が多い）

➤Blue, Green, Redで色を表現

◆ $(B, G, R) = (0, 255, 255) \rightarrow$ イエロー
 $(255, 0, 255) \rightarrow$ マゼンタ
 $(255, 255, 0) \rightarrow$ シアン
 $(255, 255, 255) \rightarrow$ ホワイト

➤色の三原色

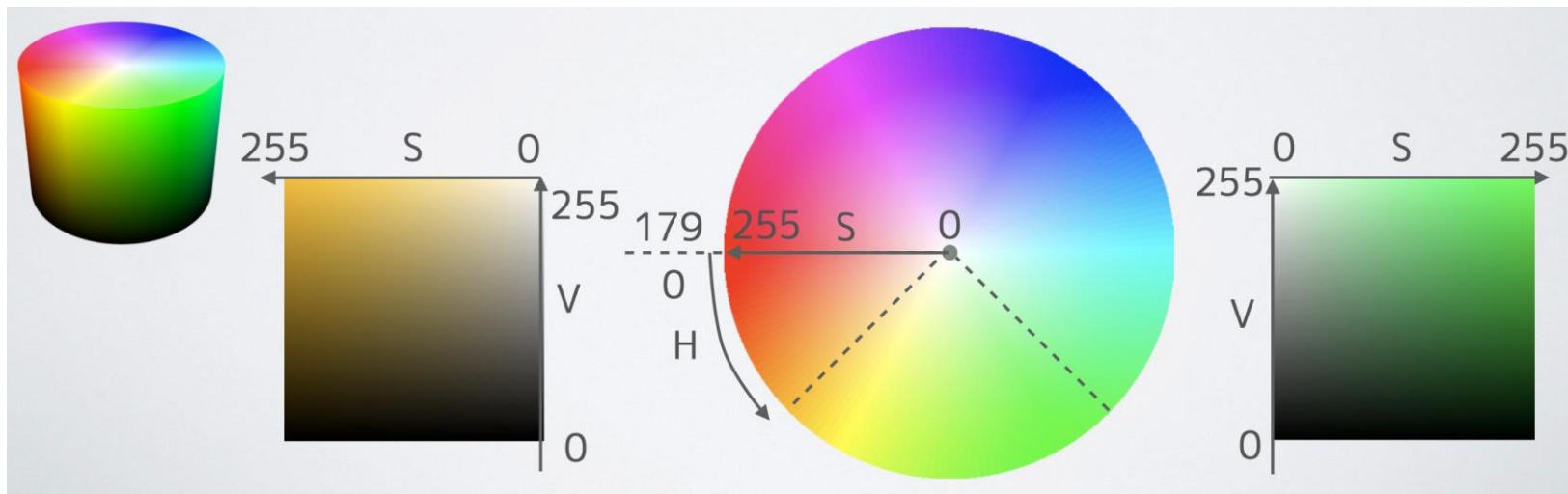
◆ディスプレイ表示に適する



デジタル画像とHSV

■HSV

- Hue, Saturation, Valueで色を表現
 - ◆ Hは0~179の180階調, sとVは0~255の256階調
- 特定色領域の抽出などの処理でRGBより適する場合あり
 - ◆ RGB・・・R, G, Bを操作する必要あり（色相はRGBの組み合わせ）
 - ◆ HSV・・・H（とs）だけで操作可能（色相はHのみ）



OpenCVでのデジタル画像の取り扱い

■画像の表現：cv::Matクラス

➤主なメンバ

- ◆int cols; //横方向の画素数
- ◆int rows; //縦方向の画素数
- ◆int channels(); //チャンネル数
- ◆int depth(); //深度情報
(CV_8U(=0), CV_32F(=5) など)
- ◆Size size(); //画像サイズ (cols, rows)
- ◆uchar *data; //画像データ配列へのポインタ

➤cv::Matクラスは画像以外の用途にも多く使用

- ◆行列, ベクトル, フィルタ

OpenCVによる画像処理（静止画）

■mc02-1.cpp

➤画像の読み書きの基本形

◆imreadで読み込み

✓必要ならリサイズして大きさ調整

◆namedWindowでウィンドウ表示

◆何かしらの画像処理

✓今回は何もしていない（コピーだけ）

◆imshowで画像表示

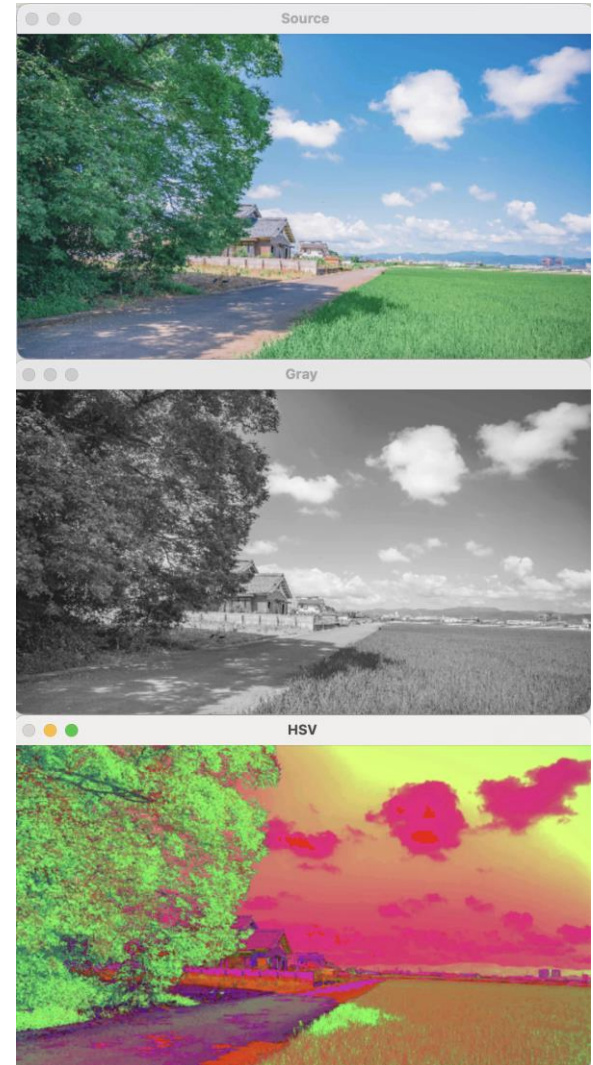
◆waitKey(0)でプログラム一時停止



mc02-1.cppの修正①（画像の色変換）

```
int main (int argc, const char* argv[])
{
    . . . . .
    //④画像処理
    // resultImage = sourceImage.clone(); // とりあえず入力画像をそのままコピー
    //画像の色変換
    Mat grayImage, hsvImage; // 画像格納用
    // BGR->GRAY (cvtColorは出力画像の配列を自動で確保してくれる)
    cvtColor(sourceImage, grayImage, COLOR_BGR2GRAY);
    // BGR->HSV
    cvtColor(sourceImage, hsvImage, COLOR_BGR2HSV);
    . . . . .
    //⑤ウィンドウへの画像の表示
    cv::imshow("Source", sourceImage);
    cv::imshow("Gray", grayImage);
    cv::imshow("HSV", hsvImage);
    cv::imshow("Result", resultImage);
    . . . . .
}
```

imshow()では
3チャンネル画像は
BGR形式が前提のため、
H→B, S→G,
V→Rと見なして画
像を表示⇒変な色に



mc02-1.cppの修正②（画素値の編集）

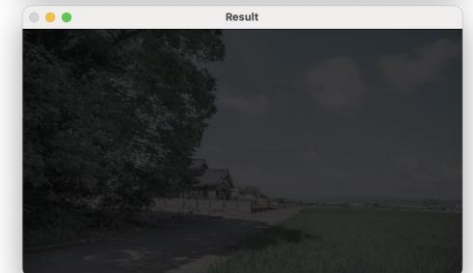
```
// BGR->HSV
cvtColor(sourcelmage, hsvImage, COLOR_BGR2HSV);
// H, S, Vをいじってみる
for(int j=0; j<hsvImage.rows; j++){
    for(int i=0; i<hsvImage.cols; i++){
        Vec3b s; // sはバイト型（8ビット）3次元ベクトル=3チャンネル画素値
        s = hsvImage.at<Vec3b>(j, i); // hsvImageの画素(i, j)の画素値の読み取り
        s[0] = 0; // Hue
        s[1] = s[1]; // Saturation
        s[2] = s[2]; // Value
        resultImage.at<Vec3b>(j, i) = s; // resultImageの画素(i, j)に画素値sを代入
    }
}
// HSV->BGR RGB色空間に戻す
cvtColor(resultImage, resultImage, COLOR_HSV2BGR);
//⑤ウィンドウへの画像の表示
cv::imshow("Source", sourcelmage);
// cv::imshow("Gray", grayImage);
// cv::imshow("HSV", hsvImage);
cv::imshow("Result", resultImage);
```



s[0]=0;
s[1]=s[1];
s[2]=s[2];



s[0]=s[0];
s[1]=s[1]/5;
s[2]=s[2];



s[0]=s[0];
s[1]=s[1]/5;
s[2]=s[2]/4;

OpenCVによる画像処理（動画）

■mc02-2.cpp

➤動画像の読み書きの基本形

◆VideoCaptureで入力ソースをセット

- ✓カメラ or ムービーファイル

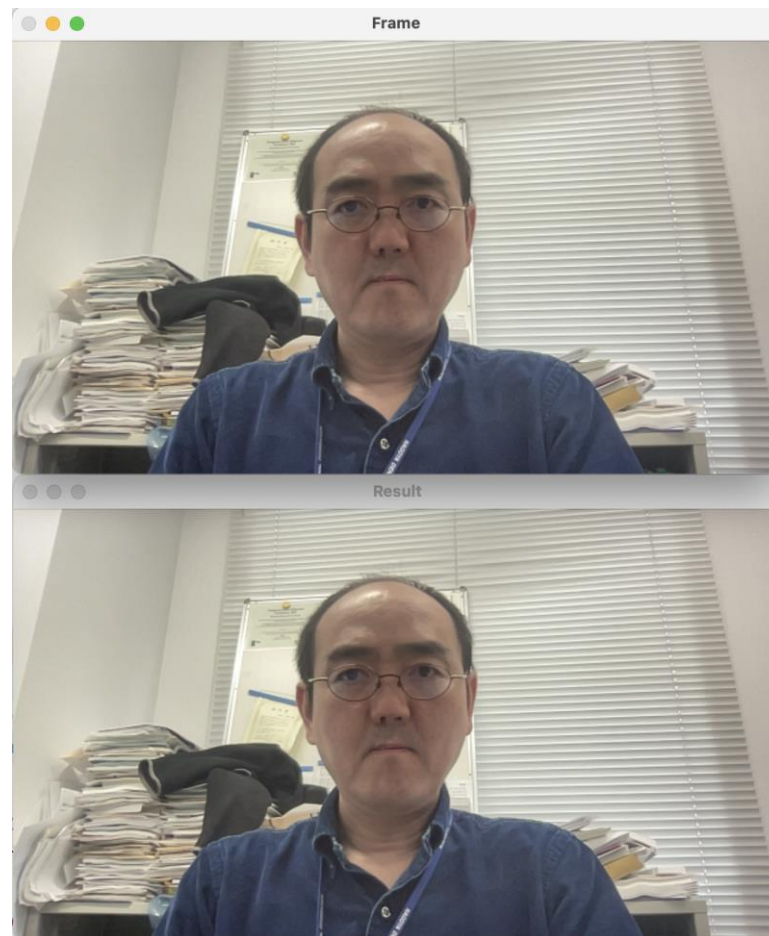
◆1フレーム取り出し

- ✓必要ならリサイズして大きさ調整

◆無限ループ生成

- 何かのキー入力で脱出できるようにしておく
- ✓1フレーム取り出し
 - 必要ならリサイズして大きさ調整
- ✓何かしらの画像処理
- ✓imshowで画像表示
- ✓処理結果画像の書き出し

◆後処理



mc02-2.cppの修正（画素値の編集）

.....

//②' 結果格納用インスタンスの生成

Mat resultImage = Mat(frameImage.size(), CV_8UC3);

Mat hsvImage;

.....

//④画像処理

// BGR->HSV

cvtColor(frameImage, hsvImage, COLOR_BGR2HSV);

// H, S, Vをいじってみる

for(int j=0; j<hsvImage.rows; j++){

for(int i=0; i<hsvImage.cols; i++){

Vec3b s; // sはバイト型（8ビット）3次元ベクトル=3チャンネル画素値

s = hsvImage.at<Vec3b>(j, i); // hsvImageの画素(i, j)の画素値の読み取り

s[0] = 0; // Hue

s[1] = s[1]; // Saturation

s[2] = s[2]; // Value

resultImage.at<Vec3b>(j, i) = s; // resultImageの画素(i, j)に画素値sを代入

}

}

// HSV->BGR RGB色空間に戻す

cvtColor(resultImage, resultImage, COLOR_HSV2BGR);



s[0]=s[0];
s[1]=s[1]/5;
s[2]=s[2];

課題02

■内臓カメラから映像を取り込み、肌色領域はそのままに、その他の領域は暗めの白黒に加工したムービーを提出せよ。

➤「課題02提出箱」に提出

➤ヒント

◆肌色のしきい値を探そう

